

安徽大学《机器视觉视觉》实验报告（5）

学号： WA2224013 专业： 机器人工程 姓名及签字： 郭义月
实验日期： 2025.6.21 实验成绩： 教师签字：

【实验名称】：基于卷积神经网络的手写数字识别

【实验目的】

1. 熟悉和掌握 matlab 基本编程环境
2. 熟悉和掌握基于 matlab 的计算机视觉编程基础
3. 通过 matlab 编程实现手写数字识别操作

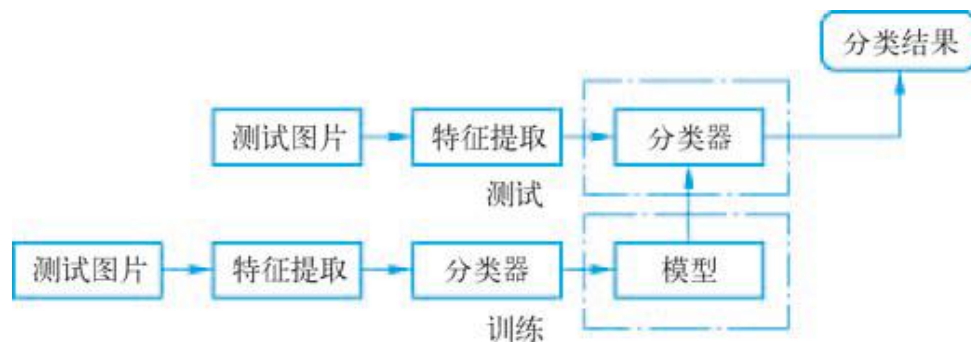
【实验内容】

案例背景

随着深度学习的迅猛发展，其应用也越来越广泛，特别是在视觉识别、语音识别和自然语言处理等很多领域都表现出色。卷积神经网络（Convolutional Neural Network, CNN）作为深度学习中应用最广泛的网络模型之一，也得到了越来越多的关注和研究。手写数字识别是图像识别学科下的一个分支，是图像处理和模式识别研究领域的重要应用之一，并且具有很强的通用性。由于手写体数字的随意性很大，如笔画粗细、字体大小、倾斜角度等因素都有可能直接影响到字符的识别准确率，所以手写数字识别是一个很有挑战性的课题。在过去的数十年中，研究者们提出了许多识别方法，并取得了一定的成果。手写体数字识别的实用性很强，在大规模数据统计如邮件分拣、人口普查、财务、税务等应用领域都有广阔的应用前景。

理论基础

本案例讲述了图像中手写阿拉伯数字的识别过程，对基于卷积神经网络的手写数字识别方法进行了简要介绍和分析，通过训练 CNN，获取模型，实现手写数字识别。



【实验代码和结果】

实验 1：加载数据

1.1 理论基础

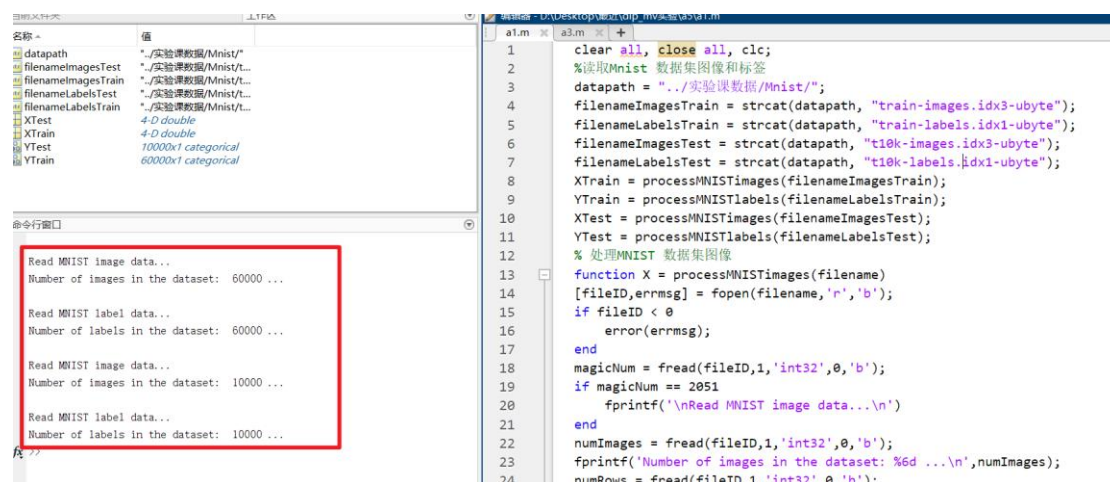
MNIST 是手写数字图片数据集，包含 60000 张训练样本和 10000 张测试样本。MNIST 数据集来自美国国家标准与技术研究所（National Institute of Standards and Technology, NIST），M 是 Modified 的缩写。训练集是由来自 250 个不同人手写的数字构成，其中 50% 是高中学生，50% 是美国人口普查局的工作人员。测试集也是同样比例的手写数字数据。每张图片由 28×28 个像素点构成，每个像素点用一个灰度值表示，这里是将 28×28 像素展开为一个一维的行向量（每行 784 个值）。图片标签为 one-hot 编码：0~9。

1.2 实验代码

```
clear all, close all, clc;
%读取 Mnist 数据集图像和标签
datapath = "../实验课数据/Mnist/";
filenameImagesTrain = strcat(datapath, "train-images.idx3-ubyte");
filenameLabelsTrain = strcat(datapath, "train-labels.idx1-ubyte");
filenameImagesTest = strcat(datapath, "t10k-images.idx3-ubyte");
filenameLabelsTest = strcat(datapath, "t10k-labels.idx1-ubyte");
XTrain = processMNISTimages(filenameImagesTrain);
YTrain = processMNISTlabels(filenameLabelsTrain);
XTest = processMNISTimages(filenameImagesTest);
YTest = processMNISTlabels(filenameLabelsTest);
```

1.3 实验结果

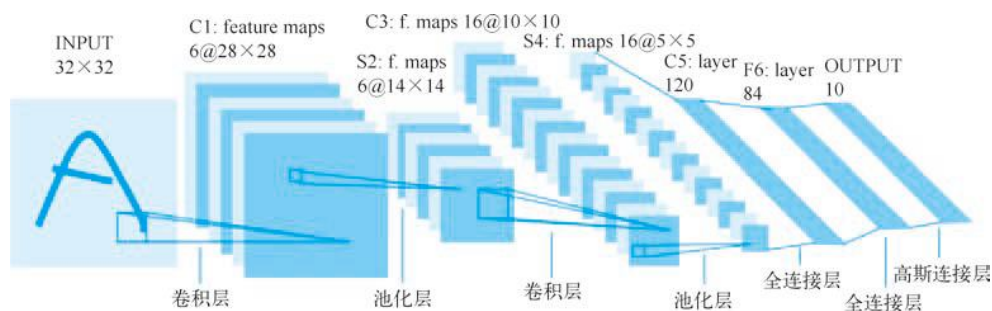
4 个文件都成功读取



实验 2: LeNet-5 网络模型

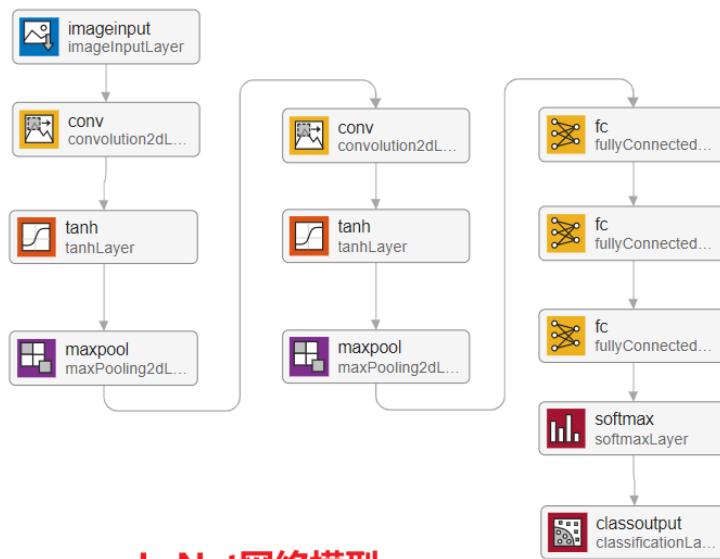
2.1 LeNet-5 网络模型

以 32×32 输入图像为例，先经卷积层 C1（6 个卷积核）提取特征得到 6 个 28×28 特征图；再由池化层 S2 对 C1 输出下采样，生成 6 个 14×14 特征图；接着卷积层 C3 用 16 个卷积核与 S2 部分特征图卷积，得到 16 个 10×10 特征图；随后池化层 S4 对 C3 输出下采样，产生 16 个 5×5 特征图；之后经卷积层 C5（含 120 个卷积核）转换为 120 维特征；再通过全连接层 F6 进一步变换为 84 维；最后经全连接输出层，借助高斯连接输出 10 维结果，对应 0-9 数字分类，实现字符识别。



2.2 LeNet-5 网络模型设计

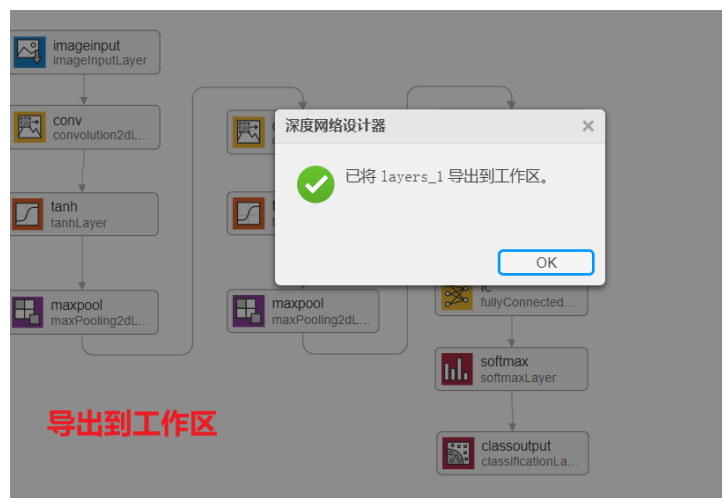
在 MATLAB 中的 App 选项卡中有一个名为 Deep Network Designer (深度网络设计师) 的 App，打开它，就可以通过拖动神经网络的组件来设计深度网络了。下图是设计的 LeNet-5 网络模型。



LeNet网络模型

首先，通过 `imageInput` 层接收图像输入，开启对图像数据的处理；接着，数据流入 `conv`（卷积）层，利用卷积核对图像进行特征提取，挖掘图像的局部特征；随后进入 `tanh` 层，借助 `tanh` 激活函数对提取的特征进行非线性变换，增加模型的表达能力；再经过 `maxpool`（最大池化）层，对特征图进行下采样，在保留关键特征的同时降低数据维度与计算量；之后重复一次“卷积→`tanh` 激活→最大池化”的操作，进一步提取和优化特征；完成特征提取后，数据进入 `fc`（全连接）层，将前面提取的特征进行整合与变换，先经两个全连接层逐步映射，最后在 `softmax` 层进行分类概率计算，输出各类别的概率；最终由 `classoutput` 层输出分类结果，实现对输入图像的识别与分类。

导出到工作区之后可以调用：



模型结构与参数已成为 MATLAB 工作区的变量。后续可在命令行查看模型细

节，借 `summary` 了解网络架构；若需优化，能用新数据、新训练策略，通过 `trainNetwork` 二次训练；接着用 `classify` 或 `predict` 对预处理后的测试图像预测，搭配真实标签算准确率、混淆矩阵评估效果；表现良好时，可嵌入 `App Designer` 做交互识别，或经 `MATLAB Compiler` 打包部署到无 `MATLAB` 环境；也能用 `save` 存为 `MAT` 文件，方便复用与协作，让模型从设计阶段迈向实际应用、持续迭代的流程。

在分析种可以查看 `LeNet` 的模型参数：



`LeNet - 5` 里的 `C1` 层，用 6 个 $5 \times 5 \times 1$ 卷积核，以步长 1、无填充的方式，对输入图像做卷积，把 32×32 图像转化为 $28 \times 28 \times 6$ 特征图，每个卷积核的权重和偏置构成可训练参数，让模型挖掘图像局部特征。

池化层用于压缩数据，`S2` 层作为平均池化层，以 2×2 窗口、步长 2 对 `C1` 输出处理，把 $28 \times 28 \times 6$ 特征图变为 $14 \times 14 \times 6$ ，在保留关键特征时降低计算量，且无额外可训练参数，就是执行固定下采样操作。后续的 `C3` 卷积层更复杂，16 个 $5 \times 5 \times 6$ 卷积核（采用部分连接策略），经步长 1、无填充卷积，输出 $10 \times 10 \times 16$ 特征图，因连接方式特殊，可训练参数计算需考虑不同输入连接组合，以此丰富特征提取维度。

`S4` 池化层继续下采样，以 2×2 窗口、步长 2 对 `C3` 输出处理，得到 $5 \times 5 \times 16$ 特征图，同样无新可训练参数。到 `C5` 层，虽名为卷积层，实际可看作全连接层，120 个 $5 \times 5 \times 16$ 卷积核（全连接到 `S4` 所有特征图），输出 $1 \times 1 \times 120$ 特征，大

量可训练参数让它深度整合前面提取的特征 。

全连接层 F6 接收 C5 输出，将 120 个输入神经元映射到 84 个输出，借助激活函数（如 $\tanh$ ）增加模型非线性表达，其可训练参数是输入输出神经元关联的权重与偏置 。最后输出层，比如对应 0 - 9 分类的 10 个输出神经元，经 Softmax 激活输出分类概率，可训练参数实现从 84 维到 10 维的映射，完成图像分类任务 。各层参数相互配合，从图像输入、特征提取、压缩，到深度整合与分类，构建起完整的深度学习网络运算流程，让模型能精准识别图像内容 。

导出的代码为：

创建深度学习网络架构

该脚本可创建具有以下属性的深度学习网络

层数: 12
连接数: 11

运行脚本以在工作区变量 layers 中创建层。
要了解详细信息，请参阅 从深度学习设计生成 MATLAB 代码。
由 MATLAB 于 2025-06-21 19:47:58 自动生成

创建层组

layers = [
imageInputLayer([28 28 1],"Name","imageinput")
convolution2dLayer([3 3],6,"Name","conv_1","Padding","same")
tanhLayer("Name","tanh_1")
maxPooling2dLayer([2 2],"Name","maxpool_1","Padding","same","Stride",[2 2])
convolution2dLayer([3 3],16,"Name","conv_2","Padding","same")
tanhLayer("Name","tanh_2")
maxPooling2dLayer([2 2],"Name","maxpool_2","Padding","same","Stride",[2 2])
fullyConnectedLayer(120,"Name","fc_1")
fullyConnectedLayer(84,"Name","fc_2")
fullyConnectedLayer(10,"Name","fc_3")
softmaxLayer("Name","softmax")
classificationLayer("Name","classoutput");

绘制层

plot(layerGraph(layers));

根据网络结构生成代码

绘制层：

代码。

由 MATLAB 于 2025-06-21 19:47:58 自动生成

创建层组

1 layers = [
2 imageInputLayer([28 28 1],"Name","imageinput")
3 convolution2dLayer([3 3],6,"Name","conv_1","Pa
4 tanhLayer("Name","tanh_1")
5 maxPooling2dLayer([2 2],"Name","maxpool_1","Pe
6 convolution2dLayer([3 3],16,"Name","conv_2","F
7 tanhLayer("Name","tanh_2")
8 maxPooling2dLayer([2 2],"Name","maxpool_2","Pe
9 fullyConnectedLayer(120,"Name","fc_1")
0 fullyConnectedLayer(84,"Name","fc_2")
1 fullyConnectedLayer(10,"Name","fc_3")
2 softmaxLayer("Name","softmax")
3 classificationLayer("Name","classoutput");

绘制层

4 plot(layerGraph(layers));

绘制层

实验 3：模型训练

3.1 训练代码：

```
layers = [  
imageInputLayer([28 28 1],"Name","imageinput")  
convolution2dLayer([3 3],6,"Name","conv_1","Padding","same")
```

```

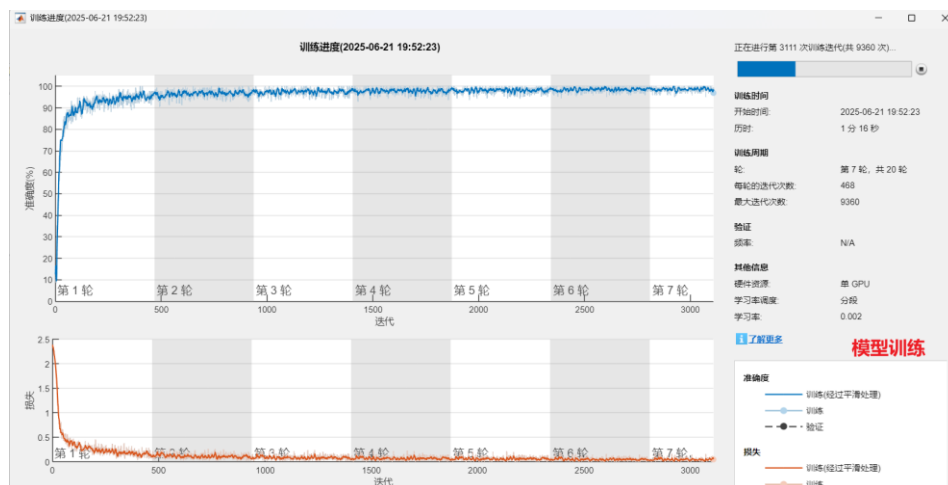
tanhLayer("Name","tanh_1")
maxPooling2dLayer([2
2],"Name","maxpool_1","Padding","same","Stride",[2 2])
convolution2dLayer([3 3],16,"Name","conv_2","Padding","same")
tanhLayer("Name","tanh_2")
maxPooling2dLayer([2
2],"Name","maxpool_2","Padding","same","Stride",[2 2])
fullyConnectedLayer(120,"Name","fc_1")
fullyConnectedLayer(84,"Name","fc_2")
fullyConnectedLayer(10,"Name","fc_3")
softmaxLayer("Name","softmax")
classificationLayer("Name","classoutput"]);

options = trainingOptions('sgdm', ... %优化器
'LearnRateSchedule','piecewise', ... %学习率
'LearnRateDropFactor',0.2, ...
'LearnRateDropPeriod',5, ...
'MaxEpochs',20, ... %最大学习整个数据集的次数
'MiniBatchSize',128, ... %每次学习样本数
'Plots','training-progress'); %画出整个训练过程
doTraining = true; %是否训练
if doTraining
    trainNet = trainNetwork(XTrain, YTrain, layers, options);
    % 训练网络, XTrain 训练的图片, YTrain 训练的标签, layers 要训练的网
    % 络, options 训练时的参数
end
save Minist_LeNet5 trainNet %训练完后保存模型
yTest = classify(trainNet, XTest); %测试训练后的模型
accuracy = sum(yTest == YTest)/numel(YTest); %模型在测试集的准确率

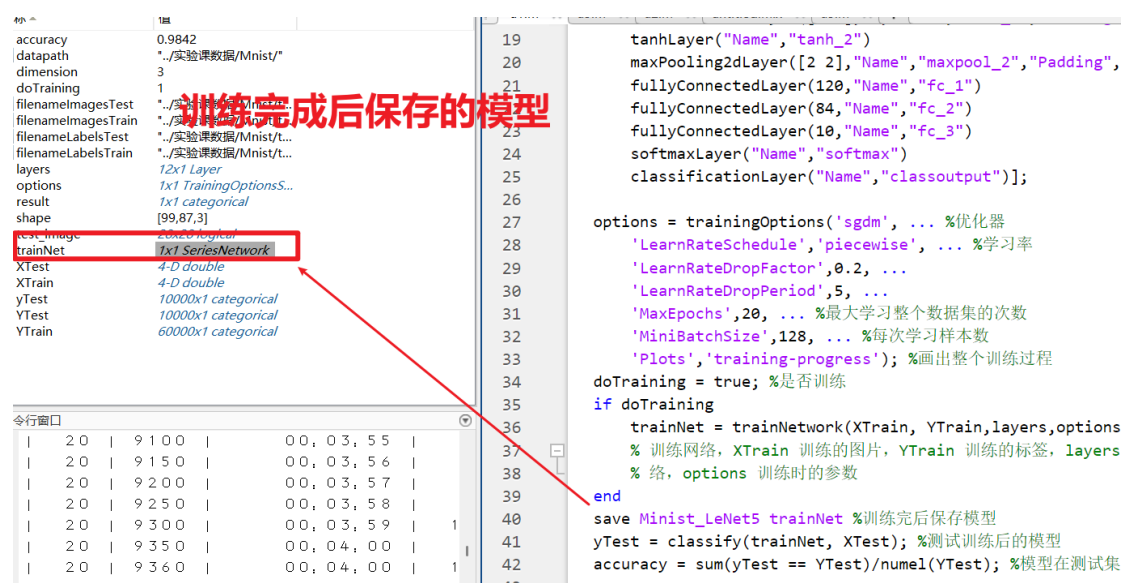
```

3.2 训练结果

开始训练:



训练完成:



实验 4：模型测试

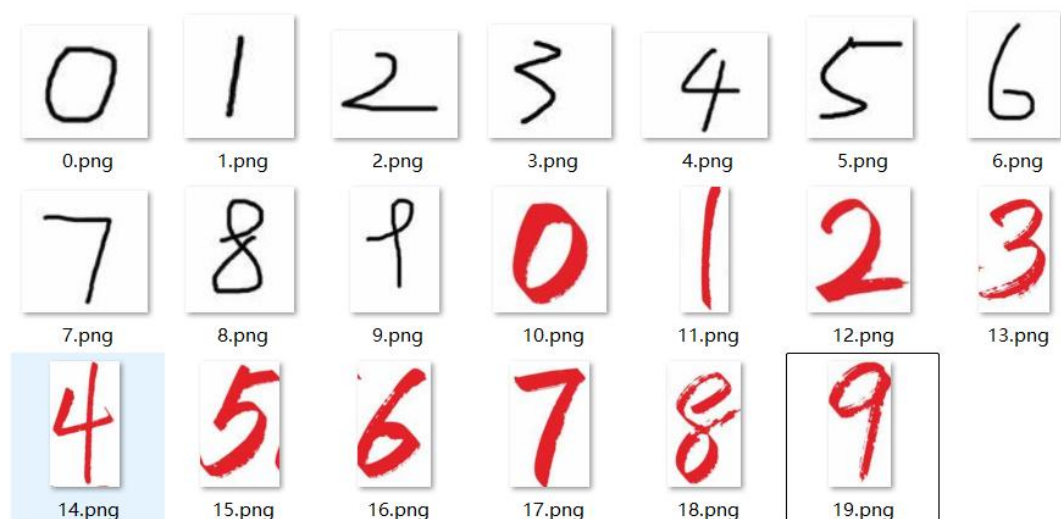
4.1 收集一些手写数字图片

在完成模型训练后，为测试模型的泛化能力，可以从网络搜集手写数字图像构建测试数据集。这些图像涵盖多样手写风格、书写笔锋与背景环境，能模拟真实应用场景中复杂多变的输入情况，以此检验模型在未见数据上的识别表现，助力评估模型对不同手写数字的适应及准确分类能力，为模型性能分析提供可靠依据。



为进一步测试模型的泛化能力，从网络搜集手写数字图像构建测试集时，特别关注到不同书写风格对模型识别的影响。此次收集了两种典型风格的手写数字样本，一类是简洁的黑色线条手写体（如 0.png - 9.png），笔画规整、形态清晰；另一类则是用红色笔书写，风格相对随性、笔触变化更丰富（如 10.png -

19.png) 。借助这样包含多样书写风格的测试数据，能检验模型在面对不同书写习惯、色彩及笔触表现时的识别稳定性，为评估模型对复杂真实场景的适应能力提供有效支撑，助力更全面地分析模型性能 。



两种不同风格的手写数字

4.2 单张图片测试

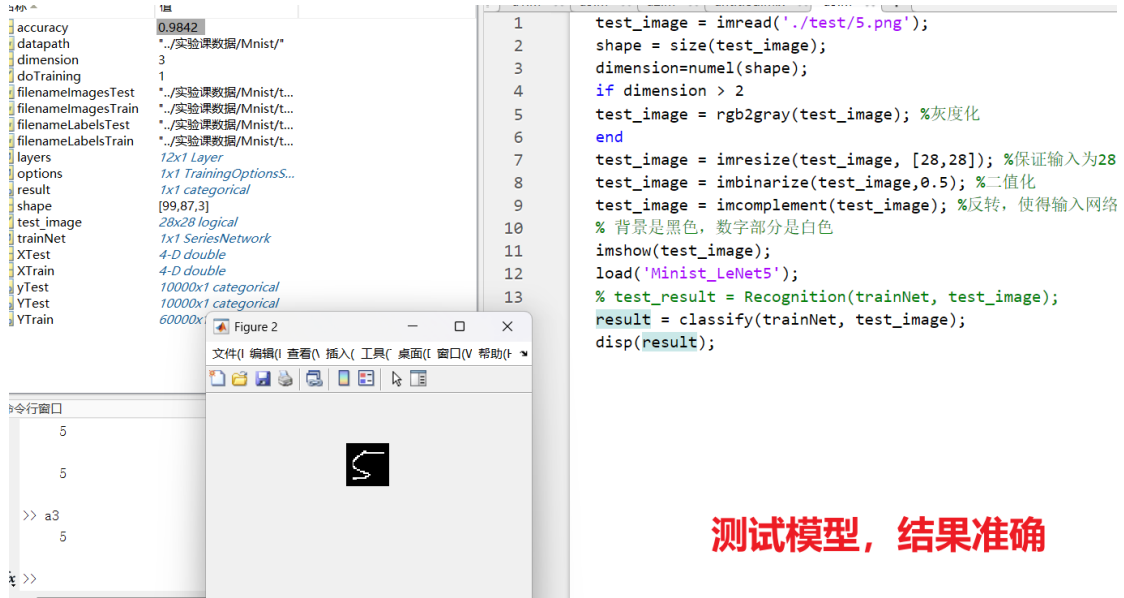
完成模型训练后，为验证其实际识别效果，选取一张手写数字图片（如 5.png ）进行测试。经图像预处理（灰度化、尺寸调整为 28×28、二值化及反转，确保背景黑、数字白的输入要求 ），输入训练好的 LeNet - 5 模型，通过 classify 函数预测，结果准确识别出数字 5 。这一测试初步验证模型在单一样本下具备正确识别不同风格手写数字的能力，为后续全面评估模型泛化性与准确率奠定基础，也让模型应用于实际手写数字识别任务的可行性得到直观体现 。

测试代码：

```
test_image = imread('./test/5.png');
shape = size(test_image);
dimension=numel(shape);
if dimension > 2
    test_image = rgb2gray(test_image); %灰度化
end
test_image = imresize(test_image, [28,28]); %保证输入为 28×28
test_image = imbinarize(test_image,0.5); %二值化
test_image = imcomplement(test_image); %反转，使得输入网络时一定要保证图片
% 背景是黑色，数字部分是白色
imshow(test_image);
```

```
load('Minist_LeNet5');
% test_result = Recognition(trainNet, test_image);
result = classify(trainNet, test_image);
disp(result);
```

测试结果:



测试模型，结果准确

4.3 多张图片测试

在完成单张手写数字图片的测试，确认模型具备初步识别能力后，进一步对文件夹内的 20 张手写数字图片开展全面测试。

实验代码:

```
figure('Position', [100, 100, 1200, 800]);
load('Minist_LeNet5');
correct_count = 0;
for i = 0:19
    image_path = ['./test/', num2str(i), '.png'];
    test_image = imread(image_path);
    shape = size(test_image);
    dimension = numel(shape);
    if dimension > 2
        test_image = rgb2gray(test_image); % 灰度化
    end
    test_image = imresize(test_image, [28, 28]); % 调整大小为 28x28
    test_image = imbinarize(test_image, 0.5); % 二值化
    test_image = imcomplement(test_image); % 反转，确保背景为黑色，数字为
    白色
    result = classify(trainNet, test_image)
    subplot(4, 5, i+1);
    imshow(test_image);
    [~, file_name, ~] = fileparts(image_path);
```

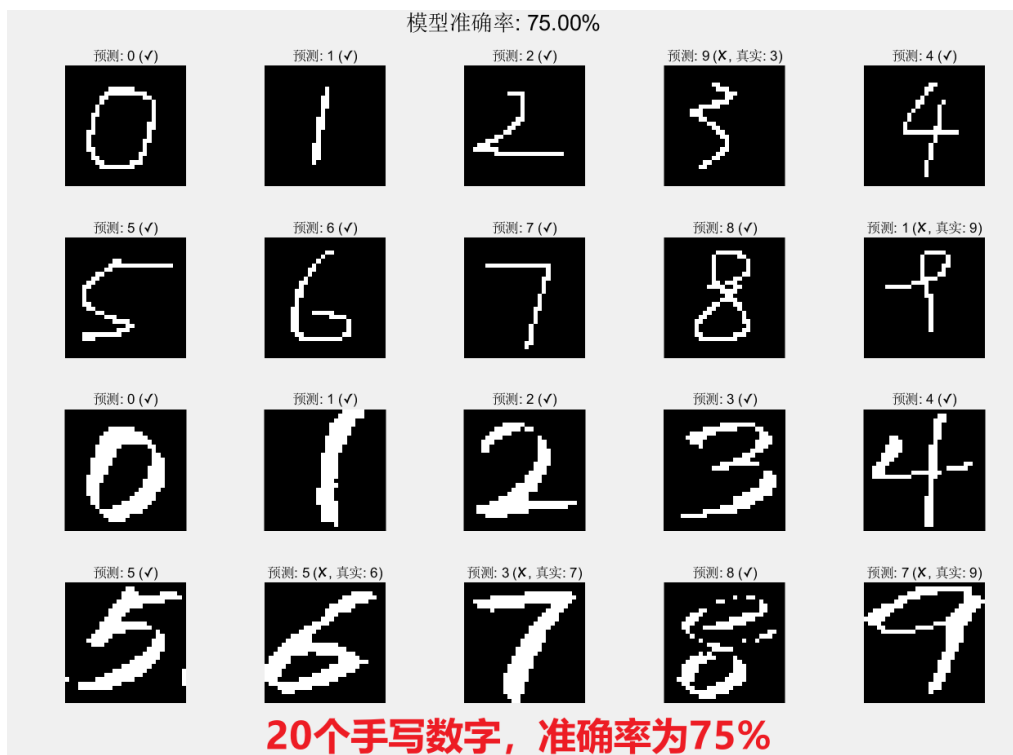
```

num = str2double(file_name);
if num < 10
    true_label = num;
else
    true_label = num - 10;
end
predicted_value = str2double(char(result));
is_correct = (predicted_value == true_label);
if is_correct
    correct_count = correct_count + 1;
    title_str = sprintf('预测: %d (✓)', predicted_value);
else
    title_str = sprintf('预测: %d (X, 真实: %d)', predicted_value,
true_label);
end

title(title_str, 'FontSize', 10);
axis off;
end
accuracy = correct_count / 20 * 100;
sgtitle(sprintf('模型准确率: %.2f%%', accuracy), 'FontSize', 16);

```

经图像预处理（灰度化、尺寸调整、二值化及反转等操作，确保输入符合模型要求），将这些涵盖不同风格、形态各异的手写数字图像依次输入训练好的 LeNet - 5 模型，借助 `classify` 函数进行预测。最终统计结果显示，20 个样本的识别准确率达 75% 。



这一测试全面检验了模型在多样本、不同手写风格场景下的泛化能力与识别性能，既展现出模型对部分手写数字的有效识别，也通过存在的错误识别案例。

【小结或讨论】

本次实验中，完成了基于卷积神经网络的手写数字识别

1. 实验全流程实现：完成基于 LeNet-5 卷积神经网络的手写数字识别，在 MATLAB 中加载 MNIST 数据集（6 万训练/1 万测试样本， 28×28 灰度图），通过 Deep Network Designer 设计网络架构，包含输入层、2 组“卷积+tanh+池化”、3 层全连接层（ $120 \rightarrow 84 \rightarrow 10$ 维）及 Softmax 分类层，实现从特征提取到数字分类的完整流程。

2. 模型训练与标准数据集表现：采用 sgdm 优化器，设置学习率分段衰减（DropFactor=0.2，周期 5 轮）、最大训练 20 轮、批量大小 128，训练后在 MNIST 测试集上准确率达 0.9842，验证模型对标准手写数字的高效识别能力。

3. 泛化能力测试与性能分析：使用 20 张不同风格手写图片（含黑色规整字体与红色随性字体）测试，经灰度化、Resize 至 28×28 、二值化及反色预处理后，模型识别准确率为 75%，表明其对非标准手写风格存在适应性局限，部分样本因笔触差异导致识别错误。

4. 实验结论与改进方向：深入理解 CNN 特征提取机制（卷积核捕捉局部特征、池化降维），掌握 MATLAB 深度学习工具链使用；针对泛化不足，可通过扩充多样化训练数据、优化预处理策略（如增强对比度）或调整网络参数（增加卷积核数量）提升模型鲁棒性。